

Growing History, Context Window & Window-Based Conversation

1. Python List Slicing — Quick Revision

Before understanding window-based conversation history, let us revise Python list slicing.

```
values = [10,20,30,40,50,60,70,80,90,100]
```

```
print(values[1:7:2])
```

```
# From begin index to end-1,
```

```
# taking every second element
```

```
print(values[1:7])
```

```
# From begin index to end-1
```

```
print(values[1:])
```

```
# From begin index to the last element
```

```
print(values[:7])
```

```
# From index 0 to end-1
```

```
print(values[-4:])
```

```
# Last 4 values
```

```
print(values[-6:])
```

```
# Last 6 values
```

Output

```
[20, 40, 60]
```

```
[20, 30, 40, 50, 60, 70]
```

```
[20, 30, 40, 50, 60, 70, 80, 90, 100]
```

```
[10, 20, 30, 40, 50, 60, 70]
```

```
[70, 80, 90, 100]
```

```
[50, 60, 70, 80, 90, 100]
```

2. General List Slicing Syntax

```
list[start:end:step]
```

Meaning

start



Where to begin

end



Stop before this index

step



How many positions to move

Example:

`values[1:7:2]`

means:

Start at index 1



Stop before index 7



Move 2 positions each time

3. Negative Slicing

Suppose:

```
values = [10,20,30,40,50,60,70,80,90,100]
```

To get the last four elements:

```
values[-4:]
```

Output:

```
[70, 80, 90, 100]
```

To get the last six:

```
values[-6:]
```

Output:

```
[50, 60, 70, 80, 90, 100]
```

This concept becomes very useful for conversation history.

4. Creating a New List with Existing Values

```
values = [10,20,30,40,50,60,70,80,90,100]
```

```
new_list = [  
    *values,  
    110,
```

```
120,  
130  
]
```

```
print(new_list)
```

Output

```
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130]
```

Here:

`*values`

unpacks all the existing list elements into the new list.

5. Growing Conversation History

Our previous chatbot program does this every time:

```
response = llm.invoke(messages)
```

Suppose `messages` contains the entire conversation:

Message 1

Message 2

Message 3

Message 4

...

Message 100

...

Message 200

Every time we call:

`llm.invoke(messages)`

all the supplied messages contribute to the current request.

6. Main Problem

Do we really need every old message for every new question?

Usually:

Not always.

Some older messages may be useful.

Some may no longer be useful.

So we need to think carefully about:

How much conversation history
should we send to the model?

7. Why Not Send Everything Forever?

There are several reasons.

8. Reason 1 — Context Window

Models have a limited context window.

Therefore:

Conversation history



Cannot grow forever
without considering



Model Context Limit

Our application should make sure that the total input fits within the model's context limit.

9. Reason 2 — Token Usage

More conversation history means more input content.

More History



More Input



More Input Tokens

10. Reason 3 — Cost

For APIs where input tokens are billed:

More Input Tokens



Potentially Higher Cost

So sending unnecessary old messages may increase cost.

11. Reason 4 — Latency

Very large requests may require more processing.

More History



Larger Input



Potentially More Processing



Potentially More Latency

12. Reason 5 — Irrelevant Information

Old conversation may no longer be useful for the current question.

Example:

100 messages ago:

Discussion about Java

Current question:

Explain Python decorators

The old Java discussion may not be required.

13. Important Rule

Our application must try to make sure the model receives:

Useful Information

+

Relevant Information

+

Input within Context Limit

14. What Can We Do?

Common strategies include:

1. Keep all history when it is small.
2. Keep only recent history.

3. Summarize older history.
4. Store important information separately.
5. Retrieve only relevant past information.

Today we mainly focus on:

Recent History Window

15. What Is a Conversation Window?

A conversation window means keeping or sending only the **most recent part of the conversation**.

Suppose full history contains:

Message 1

Message 2

Message 3

Message 4

Message 5

Message 6

Message 7

Message 8

Message 9

Message 10

Suppose window size is:

4 messages

Then we send:

Message 7

Message 8

Message 9

Message 10

The older messages remain outside the current window.

16. Window Concept

Complete History

M1

M2

M3

M4

M5

M6

M7

M8

M9

M10



Window Size = 4



M7

M8

M9

M10

17. Python Makes Windowing Easy

Students already know list slicing.

Suppose:

```
messages = [  
    "M1",  
    "M2",
```

```
"M3",  
"M4",  
"M5",  
"M6"  
]
```

To get the last four:

```
recent_messages = messages[-4:]
```

Result:

M3

M4

M5

M6

18. Conversation Window Using Slicing

General formula:

```
recent_messages = messages[-N:]
```

where:

N = Number of recent messages required

Example:

```
recent_messages = messages[-6:]
```

means:

Keep the last 6 messages

19. Important — Messages vs Turns

Suppose one normal conversation turn contains:

HumanMessage

+

AIMessage

That means:

1 Turn

↓

2 Messages

Therefore:

3 recent turns

↓

Approximately 6 messages

So:

`messages[-6:]`

roughly keeps:

3 Human + AI turns

assuming the conversation normally alternates between human and AI messages.

20. Messages vs Tokens

Very important:

Message Count

≠

Token Count

One message may contain:

5 words

Another message may contain:

500 words

Both are:

1 message

but their token usage is very different.

So:

6 messages

does not always mean the same number of tokens.

21. Interactive Window-Based Chatbot

Now modify our chatbot so that:

Application stores complete history



But sends only recent history



To the model

22. Program — Window-Based Chatbot

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
# Complete conversation history
messages = []

# Maximum recent messages sent to model
MAX_MESSAGES = 6

while True:

    user_input = input(
        "\nYou: "
    )

    if user_input.lower() == "exit":

        print("Chat ended.")

        break
```

```
# Add user message to complete history
```

```
messages.append(  
    HumanMessage(  
        content=user_input  
    )  
)
```

```
# Select recent conversation
```

```
recent_messages = messages[  
    -MAX_MESSAGES:  
]
```

```
# Send only recent history
```

```
response = llm.invoke(  
    recent_messages  
)
```

```
print(  
    "AI:",  
    response.content  
)
```

```
# Store AI response in complete history  
messages.append(  
    response  
)
```

23. Most Important Part of the Program

```
recent_messages = messages[  
    -MAX_MESSAGES:  
]
```

Suppose:

MAX_MESSAGES = 6

Then:

```
recent_messages = messages[-6:]
```

This selects only the latest six messages.

24. Complete History vs Recent History

Suppose:

Complete History = 20 messages

and:

MAX_MESSAGES = 6

Then:

Stored by Application



20 messages

But:

Sent to Model



Last 6 messages

Very important:

20 Stored

but

6 Sent

25. Are Old Messages Deleted?

In this approach:

No

Because:

messages

still contains the complete history.

We only create another list:

```
recent_messages = messages[-6:]
```

26. Stored History vs Sent History

This is one of the most important concepts.

STORED HISTORY



All messages maintained

by the application

versus:

SENT HISTORY

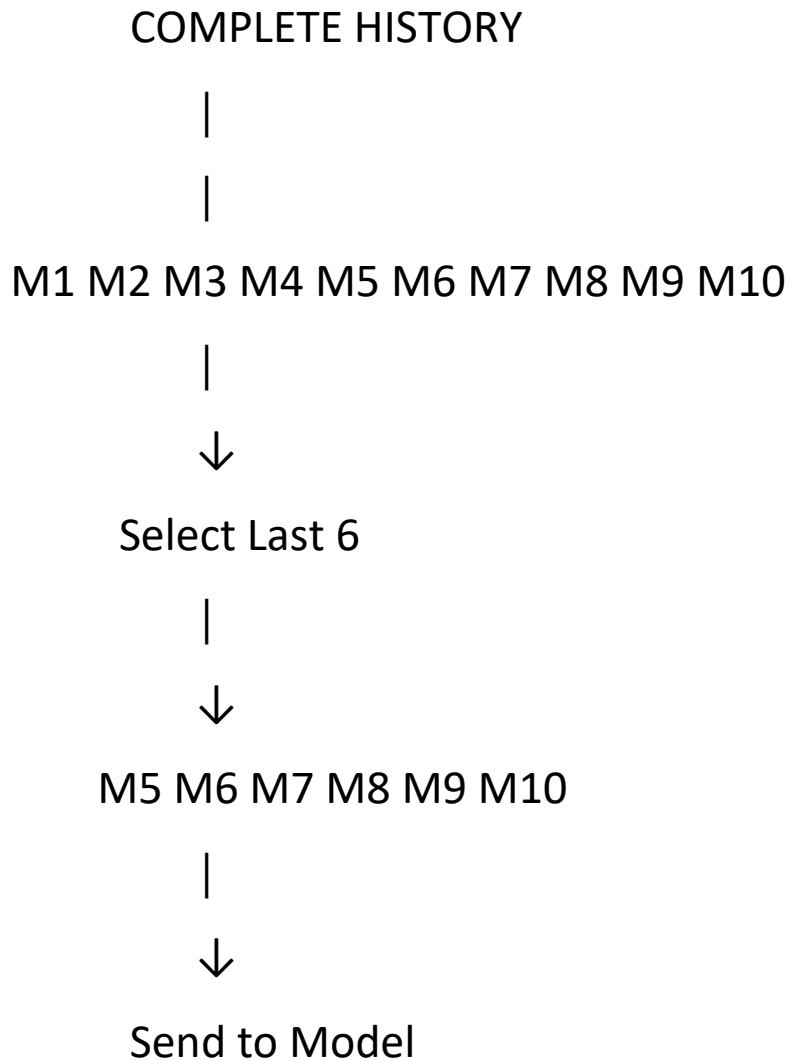


Messages actually supplied

to the model

These do not have to be the same.

27. Master Diagram



28. Program — Print Complete History and Sent History

```
print("\nComplete Stored History:")
```

```
print("-----")
```

```
for message in messages:
```

```
    print(
        type(message).__name__,
        ":",
        message.content
    )
```

```
print("\nHistory Sent to Model:")
```

```
print("-----")
```

```
for message in recent_messages:
```

```
    print(
        type(message).__name__,
        ":",
        message.content
    )
```

)

This helps students clearly see the difference between:

Stored History

and:

History Sent to Model

29. Important Question

Are we deleting old messages from messages?

Answer:

No

Because:

```
recent_messages = messages[-6:]
```

creates/selects the recent portion.

The original:

`messages`

still keeps the full list.

30. Approach 1 — Keep Full History, Send Recent History

```
recent_messages = messages[-6:]
```

Here:

messages



Complete history remains

and:

recent_messages



Only last 6 messages

Concept

Stored History = Full

Sent History = Recent Window

31. Approach 2 — Actually Trim the List

Another approach:

```
messages = messages[-6:]
```

Now the original list itself is reduced.

Old messages are removed from the current Python list.

Concept

Before

messages



20 messages

After:

messages = messages[-6:]

messages



6 messages

32. Difference Between the Two Approaches

Approach	Code	What Happens?
Keep full history	<code>recent_messages = messages[-6:]</code>	Original history stays complete
Trim history	<code>messages = messages[-6:]</code>	Original list itself becomes smaller

33. Easy Memory

Approach 1

`recent_messages = messages[-6:]`

means:

SELECT recent history

Approach 2

messages = messages[-6:]

means:

REPLACE full list

with recent history

34. Why Approach 1 Can Be Useful

If we keep full history separately:

Complete History



Available to Application

but send only:

Recent Window



To Model

then the application can still use older messages later for other purposes.

35. Important Problem with Simple Windowing

Suppose:

Message 1:

My name is Durga.

After many messages:

Message 30:

What is my name?

Suppose our recent window contains only:

Messages 25 to 30

Then:

"My name is Durga"

may be outside the current window.

So the model may no longer receive that information.

36. Windowing Solves One Problem but Creates Another

Windowing solves:

Too Much History

But it may create another problem:

Important Old Information

May Be Missing

from Current Context

37. Example

Complete history:

M1 → My name is Durga

M2

M3

M4

...

M25

M26

M27

M28

M29

M30 → What is my name?

Suppose:

```
recent_messages = messages[-6:]
```

Model receives only:

M25

M26

M27

M28

M29

M30

It does not receive:

M1 → My name is Durga

So it may not know the answer from the supplied recent history.

38. Very Important Trade-Off

Large History Window

LARGE HISTORY WINDOW



More Context

But:

More Context



More Tokens

Small History Window

SMALL HISTORY WINDOW



Fewer Messages



Fewer Tokens

But:

May Lose

Important Old Context

39. Large Window vs Small Window

Large Window	Small Window
More previous context	Less previous context
More input content	Less input content
More token usage	Fewer tokens
May increase cost	May reduce cost
May increase latency	May reduce input size
Less chance of losing recent context	Greater chance of losing older useful information

40. Is There a Perfect Window Size?

No.

There is no magic window size that is best for every application.

The correct size depends on:

1. Application
 2. Type of conversation
 3. Model context window
 4. Importance of old information
 5. Cost requirements
-

41. How Can We Handle Important Old Information?

Later, more advanced designs can use:

1. Summaries
 2. Persistent state
 3. Separate user profile information
 4. Databases
 5. Retrieval
 6. LangGraph state/checkpoints
-

42. Strategy 1 — Summary

Older messages can be summarized.

Instead of sending:

50 old messages

we may keep a shorter summary of important information.

Concept:

Old Conversation



Summary



Use Summary + Recent Messages

43. Strategy 2 — Separate Important Information

Some important information may be stored separately.

Example:

User Name

User Preferences

Account Information

Important Application State

So it does not depend only on the recent message window.

44. Strategy 3 — Retrieval

Instead of sending all history:

All Past Information

the application can retrieve only the relevant information.

Concept:

Current Question



Find Relevant Old Information



Send Relevant Information



Model

45. Full Window-Based Conversation Flow

User Input



HumanMessage



Append to Complete History



messages



Select Recent Window



messages[-MAX_MESSAGES:]



Send Recent History to Model



AI Response



Append AI Response
to Complete History



Next Turn

46. Master Program Logic

```
messages = []
```

```
MAX_MESSAGES = 6
```

```
while True:
```

```
user_input = input("You: ")

messages.append(
    HumanMessage(
        content=user_input
    )
)

recent_messages = messages[
    -MAX_MESSAGES:
]

response = llm.invoke(
    recent_messages
)

messages.append(
    response
)
```

47. Important Key Points

1. Conversation history keeps growing during a long conversation.
2. Sending the complete history every time may not always be necessary.
3. Models have a limited context window.
4. More history means more input content.
5. More input content generally means more input tokens.
6. More tokens may increase API cost.
7. Large inputs may also increase latency.
8. Old messages may become irrelevant.
9. A recent history window sends only the latest part of the conversation.
10. Python list slicing makes recent-window selection easy.
11. `messages[-6:]` means the last six messages.
12. One normal conversation turn usually contains one `HumanMessage` and one `AIMessage`.
13. Message count and token count are not the same.
14. Stored history and history sent to the model can be different.

15. `recent_messages = messages[-6:]` does not delete old history.
 16. `messages = messages[-6:]` trims the actual list.
 17. Small windows reduce history but may lose important old context.
 18. Large windows preserve more context but use more tokens.
 19. There is no single best window size for every application.
-

48. Common Mistake — Messages and Turns

Do not assume:

6 messages = 6 conversation turns

Normally:

1 turn

=

1 HumanMessage

+

1 AIMessage

Therefore:

6 messages

≈

3 complete turns

assuming normal human/AI alternation.

49. Common Mistake — Messages and Tokens

Do not assume:

10 messages

=

Fixed number of tokens

Wrong.

Message lengths can be very different.

Short Message



Few Tokens

Long Message



Many Tokens

50. Common Mistake — Thinking Slicing Deletes Old History

This:

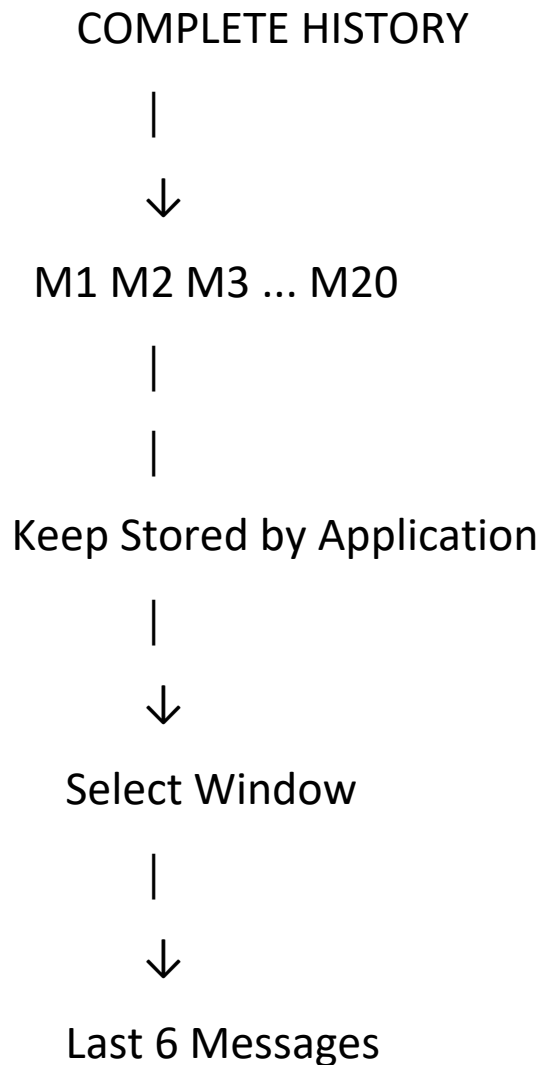
```
recent_messages = messages[-6:]
```

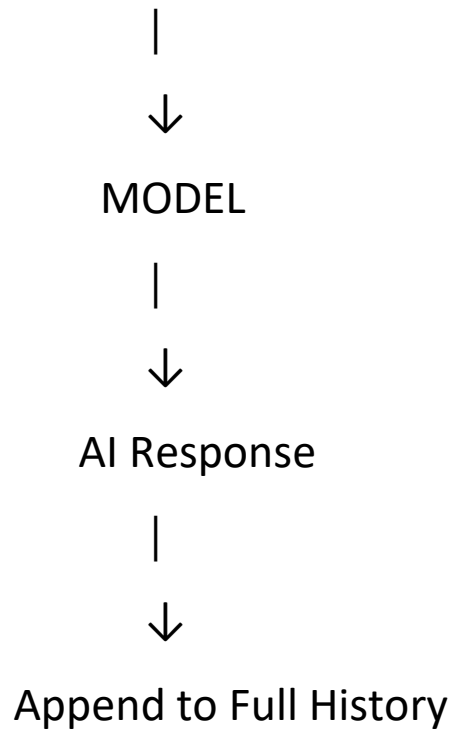
does not delete the old items from:

messages

It simply selects the recent portion into another list.

51. Final Master Diagram





52. Easy Memory Formula

FULL HISTORY



SLICE



RECENT HISTORY



SEND TO MODEL

53. Stored History vs Sent History

Remember:

Stored History

≠

Sent History

Example:

Stored = 20 Messages

Sent = Last 6 Messages

54. Windowing Trade-Off

LARGE WINDOW



More Context



More Tokens

versus:

SMALL WINDOW



Fewer Tokens



May Lose Old Context

55. One-Line Conclusion

Window-based conversation history keeps the complete conversation if required but sends only a recent portion to the model, helping control context size, token usage, cost and latency while accepting the risk that some important older information may fall outside the current window.

Final Memory Line

Growing History



Select Recent Messages



messages[-N:]



Send Recent Window



Model

And remember:

DEPEND ON ALL HISTORY?



Not Always

USEFUL RECENT HISTORY



Window-Based Conversation